INFOTACT SOLUTIONS & CO.

Infotact Technical Internship Program: Data Science & Machine Learning Project Specifications and Evaluation Criteria

The following document establishes the technical requirements, algorithmic directives, and sprint-based modeling roadmaps for three distinct Data Science and Machine Learning (DS/ML) projects. Designed for execution within a fast-paced technology environment in Bengaluru, Karnataka, these specifications serve as comprehensive blueprints for AI and ML interns entering the infotact startup ecosystem. The documentation provides structured frameworks intended to guide the training of predictive models, handling of complex datasets, and deployment of machine learning pipelines across three high-growth industry niches: Government/Public Sector, Agriculture, and Restaurant/Hospitality.

The primary directive of this documentation is to standardize the machine learning lifecycle, ensuring that all deliverables align with enterprise-grade data architecture patterns, rigorous version control protocols, and robust mathematical evaluation standards. Consequently, ML interns must operate at a professional level, demonstrating proficiency in data preprocessing, statistical problem-solving, and model optimization.

This document outlines the requisite tasks, technological paradigms, and evaluation criteria for successful internship completion. It is imperative to note that the assessment of these projects relies heavily on the transparency and consistency of the development lifecycle. **The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.**

Project 1: Government & Public Sector - AI-Driven Citizen Grievance & Sentiment Analysis System

Executive Problem Statement

Effective complaint resolution is a crucial component of governance, but current methods for filing and monitoring public grievances are often disjointed, sluggish, and ineffective. Citizens frequently navigate a maze of siloed departments, resulting in poor interdepartmental communication and severe delays. Additionally, government workers must spend hours manually reading through feedback, which leads to backlogs and a lack of prioritization for critical or urgent issues.

The objective of this project is to architect an AI-powered Natural Language Processing (NLP) system that automatically ingests citizen feedback, categorizes the complaints into relevant government departments (e.g., water, electricity, roads), and performs sentiment analysis to prioritize issues based on urgency and emotional tone.

Business Objectives and Key Performance Indicators

The strategic vision for this platform is to dramatically reduce the turnaround time for grievance redressal and improve governmental transparency. By automating the tagging of topics and sentiment, processes that typically take human analysts weeks can be executed in minutes.

Success will be quantified using standard NLP evaluation metrics. The intern must achieve high categorization accuracy and optimize the Macro $F1$-score for sentiment classification, ensuring the model correctly identifies minority classes (such as "Urgent/Critical" emergencies). The system must effectively eliminate the "black box" of government feedback by creating a structured, prioritized queue for civic officials.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Civic Official	Rapid triage of community issues without manual reading of every submission.	Views a dashboard of AI-tagged complaints, instantly identifying high-priority issues that require immediate dispatch.
Citizen / User	Quick acknowledgment and routing of their specific community issue.	Submits a raw text description of an issue, which the AI instantly routes to the correct department without manual form-filling.
Policy Maker	Macroscopic understanding of public sentiment on recent initiatives.	Analyzes aggregate sentiment trends across different municipalities to gauge the popularity or friction of public projects.

Minimum Viable Product Specifications

The foundational feature is the Text Preprocessing and EDA pipeline. The intern will handle raw, unstructured text data, requiring rigorous cleaning techniques such as tokenization, lemmatization, and the removal of stopwords.

The core deliverable is the Dual-Output NLP Engine. The intern must implement models capable of both multi-class classification (predicting the target department) and sentiment analysis (scoring the urgency/frustration level). The MVP should start with baseline models like

Naive Bayes or Support Vector Machines (SVM) before advancing to transformer-based architectures for contextual understanding.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Data Processing & NLP	Python, NLTK, spaCy	Essential libraries for handling text tokenization, part-of-speech tagging, and named entity recognition.
Predictive Modeling	Scikit-Learn, HuggingFace (BERT)	Scikit-learn provides fast baseline algorithms, while HuggingFace Transformers offer state-of-the-art context awareness for nuanced citizen feedback.
Model Serving	FastAPI	A modern, high-performance web framework used to expose the trained NLP models as RESTful APIs for integration with government portals.

Four-Week Engineering Roadmap

Week 1: Data Collection, Text Cleaning, and EDA

The project initiates with the establishment of the Git repository. The intern will ingest a dataset of public grievances or civic feedback (e.g., synthetic 311 service request data). The primary task is rigorous text preprocessing: converting text to lowercase, removing special characters/URLs, and applying lemmatization. The intern must generate Word Clouds and n-gram frequency distributions to visualize the most common civic complaints. All EDA must be documented thoroughly in Jupyter Notebooks.

Week 2: Topic Modeling and Department Categorization

Week two focuses on routing the complaints. The intern will convert the cleaned text into numerical vectors using TF-IDF (Term Frequency-Inverse Document Frequency) or word embeddings (Word2Vec). They will then train a supervised classification model (such as Logistic Regression or Random Forest) to automatically map a text input to a specific department label (e.g., "Sanitation", "Transport"). Cross-validation must be used to ensure the model generalizes well.

Week 3: Sentiment Analysis and Urgency Scoring

The third week tackles the emotional tone of the feedback. The intern will train a secondary model—or fine-tune a pre-trained Transformer like BERT or RoBERTa—to classify the sentiment of the text as Positive, Neutral, Negative, or Critical/Urgent. The system must assign a mathematical priority score to each ticket based on the severity of the sentiment detected.

Week 4: API Development, Evaluation, and Final Delivery

The final week is dedicated to deployment and evaluation. The intern will evaluate the models using Confusion Matrices and Classification Reports. The models will then be serialized and wrapped in a FastAPI application. The API must accept a JSON payload containing a citizen's raw text complaint and return the predicted department and priority score. The final submission must include a fully documented GitHub repository reflecting daily commits and architectural decisions.

Project 2: Agriculture & Smart Farming - Computer Vision for Crop Disease Detection

Executive Problem Statement

Global agriculture faces massive challenges as plant diseases and pests threaten food security and crop yields, with diseases contributing to massive yield losses annually. Traditional manual monitoring of crop health relies on farmers visually inspecting fields; this process is labor-intensive, prone to human error, and often catches outbreaks too late. Waiting too long to take action leads to devastating crop loss or the indiscriminate, environmentally harmful overuse of chemical pesticides.

The objective of this project is to develop a deep learning-based image classifier utilizing Computer Vision. The model must analyze images of plant leaves to detect early signs of disease, enabling timely, targeted interventions and supporting precision agriculture objectives.

Business Objectives and Key Performance Indicators

The strategic vision for this tool is to democratize expert agricultural knowledge, giving local farmers instant, smartphone-accessible diagnostic capabilities. Early and accurate detection mitigates damage and ensures sustainable food production.

Success will be measured by the model's ability to accurately distinguish between healthy and diseased plants across multiple classes. The intern must evaluate the model using Accuracy, Precision, and Recall metrics. High Recall is particularly critical; failing to identify a diseased leaf (false negative) allows the infection to spread across the field.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Local Farmer	Fast, reliable diagnosis without requiring expert botanical knowledge.	Captures a photo of a suspicious leaf via a mobile interface and receives an instant AI diagnosis and treatment recommendation.
Agronomist	Scalable monitoring of large geographic farming zones.	Utilizes the aggregated model predictions to track disease outbreak patterns across regions.
Machine Learning Engineer	High-quality image preprocessing and robust model generalization.	Augments limited image datasets and tunes Convolutional Neural Networks to handle varying field lighting conditions.

Minimum Viable Product Specifications

The foundational feature is the Image Preprocessing and Augmentation pipeline. Agricultural images suffer from environmental variability (lighting, dust, background noise). The intern must standardize image sizes, normalize pixel values, and apply data augmentation (rotations, flips, contrast adjustments) to artificially expand the training dataset.

The core deliverable is the Convolutional Neural Network (CNN). The intern must build a custom CNN architecture from scratch, and subsequently apply Transfer Learning using advanced pre-trained models (such as ResNet50 or EfficientNet) to achieve enterprise-grade accuracy.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale

Image Processing	Python, OpenCV, PIL	OpenCV is the industry standard for programmatic image manipulation, resizing, and color-space conversion.
Deep Learning Framework	TensorFlow/Keras or PyTorch	These frameworks provide the specialized tensor operations and GPU acceleration required to train deep convolutional layers.
Dataset	PlantVillage Dataset	The gold standard open-source repository containing tens of thousands of labeled images of healthy and diseased crop leaves.

Four-Week Engineering Roadmap

Week 1: Image Acquisition, EDA, and Preprocessing

The project begins with downloading a subset of the PlantVillage dataset. The intern will perform Exploratory Data Analysis by plotting sample images from each class and analyzing the class distribution to check for imbalances. The intern must build an automated preprocessing pipeline that resizes images to a standard dimension (e.g., 224×224), normalizes pixel arrays, and splits the data into strictly separated train, validation, and test sets.

Week 2: Custom CNN Architecture and Baseline Training

Week two focuses on building a baseline deep learning model. The intern will construct a custom Convolutional Neural Network containing sequential Conv2D layers, MaxPooling layers, and dense fully-connected layers. The intern must configure categorical cross-entropy loss and train the model, actively monitoring the training and validation loss curves to detect and mitigate overfitting using Dropout and Early Stopping techniques.

Week 3: Transfer Learning and Hyperparameter Optimization

The third week introduces advanced computer vision techniques. To improve upon the baseline model, the intern will implement Transfer Learning by importing a pre-trained architecture like ResNet50 or MobileNet, freezing the base layers, and fine-tuning the classification head for the specific crop diseases. The intern will experiment with hyperparameter tuning (learning rates, batch sizes) to push the accuracy above 90%.

Week 4: Evaluation, Inference, and Deployment

The final sprint is dedicated to rigorous evaluation. The intern must generate a Confusion Matrix to analyze misclassifications between visually similar diseases. The final model weights must be saved, and the intern will write an inference script that takes a raw, unseen image path as input and outputs the predicted disease and confidence score. The repository must be fully documented, with a commit history showing the iterative improvement of the network architectures.

Project 3: Food & Restaurant Services - AI Demand Forecasting and Inventory Optimization

Executive Problem Statement

The restaurant and hospitality industry operates on razor-thin margins and deals with highly perishable inventory. Traditional inventory management relies on static spreadsheets, gut feelings, or basic historical averages, which fail to account for the dynamic factors influencing customer demand. This results in massive inefficiencies: over-ordering leads to expensive food waste and spoilage, while under-ordering results in stockouts and lost revenue.

The objective of this project is to build an AI-powered Demand Forecasting model for a simulated restaurant or food delivery chain. By analyzing historical point-of-sale (POS) data and external variables, the model will predict future daily sales volume for specific menu items, allowing managers to optimize their supply chain and labor scheduling.

Business Objectives and Key Performance Indicators

The strategic vision is to transition restaurant operations from reactive guesswork to proactive, data-driven planning. Accurate demand forecasting directly translates to a 20-30% reduction in food waste and warehousing costs.

Success will be measured strictly using regression metrics. The intern must minimize the Mean Absolute Error (MAE) and Root Mean Square Error (RMSE) of the predictions. The model must successfully capture complex patterns such as weekend spikes, seasonal fluctuations, and long-term business growth.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow

Restaurant Manager	Accurate insights on how much raw ingredient to prep for the upcoming week.	Reviews the daily demand predictions to adjust purchase orders and reduce perishable food waste.
Supply Chain Director	Macroscopic visibility across multiple franchise locations to consolidate logistics.	Analyzes aggregated forecasts to negotiate better bulk pricing with wholesale food distributors.
Data Scientist	Clean time-series tracking and robust feature engineering frameworks.	Integrates external APIs (e.g., weather or local event data) to improve the baseline model's predictive power.

Minimum Viable Product Specifications

The foundational feature is the Time-Series Data Wrangling module. The intern must transform raw transactional logs into structured, chronological datasets, aggregating sales by day or hour, and meticulously handling missing dates or outlier events (e.g., extreme spikes due to holidays).

The core deliverable is the Predictive Forecasting Engine. The intern will move beyond simple moving averages and implement advanced machine learning algorithms (such as XGBoost, Random Forest Regressor, or Prophet) capable of learning non-linear relationships between the date, external features, and the target sales volume.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Data Processing	Python, Pandas, NumPy	Essential for complex time-series aggregations, rolling window calculations, and feature engineering.

Forecasting Algorithms	Scikit-Learn, XGBoost, Prophet	Prophet handles daily seasonality exceptionally well, while XGBoost excels at capturing non-linear feature interactions.
Data Visualization	Matplotlib, Plotly	Required to visually compare the predicted demand curve against the actual historical sales curve.

Four-Week Engineering Roadmap

Week 1: Data Ingestion and Time-Series EDA

The project initiates with acquiring a retail/restaurant sales dataset. The intern will clean the data, ensuring the datetime index is properly formatted and continuous. The intern must perform deep Exploratory Data Analysis (EDA) specifically tailored for time-series: plotting the overall sales trend, decomposing the data to identify seasonality (weekly and monthly patterns), and analyzing autocorrelations to understand how past sales influence future demand.

Week 2: Advanced Feature Engineering

Week two focuses on creating the inputs that will give the model predictive power. The intern will engineer chronological features (day of the week, month, is_weekend, is_holiday). Crucially, the intern must create "lag features" (e.g., sales from 7 days ago) and "rolling window statistics" (e.g., 14-day moving average). The dataset must then be split sequentially (e.g., training on the first 10 months, testing on the last 2 months) to prevent data leakage from the future into the past.

Week 3: Model Training and Selection

The third week is dedicated to predictive modeling. The intern will establish a baseline using a simple model (like Linear Regression) before training advanced ensemble models (Random Forest, XGBoost). The intern will experiment with feeding the engineered features into the model to predict the next day's sales volume. Hyperparameter tuning must be executed using Time-Series Cross-Validation to optimize the model's accuracy on unseen future data.

Week 4: Evaluation, Feature Importance, and Business Reporting

The final week focuses on business translation and evaluation. The intern will calculate the MAE and RMSE to quantify the model's error margin. They will extract Feature Importance scores to explain to stakeholders which variables (e.g., day of the week vs. recent sales trends) drive the most demand. The final deliverable must include a visualized forecast overlaying predictions against actuals, pushed to a pristine GitHub repository containing the entire data pipeline.

Universal Machine Learning and Version Control Guidelines

To simulate a rigorous, enterprise-grade AI research and development environment indicative of top-tier deep-tech companies in Bengaluru, strict adherence to modern MLOps practices and version control is an absolute requirement. The core tenet of this internship program is continuous, transparent, and high-quality contribution. Machine learning is an iterative, experimental process, and your version control history must reflect this.

Mandatory Evaluation Protocol

The evaluation of the intern's performance and the final project submission is governed by a strict, non-negotiable directive regarding version control consistency.

The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.

A project submitted with a compressed commit history—for example, uploading the entire Jupyter Notebook, all datasets, and final model weights in a massive, monolithic push during the final week or on the final day—will result in immediate disqualification. Professional data science is a continuous process of iteration, and the version control history must accurately reflect daily data cleaning progress, algorithmic pivots, hyperparameter tuning, and debugging efforts.

GitHub Operational Standards and Data Security Practices

To pass the technical evaluation, interns must internalize and strictly adhere to the following Git and GitHub operational standards.

1. Commit Frequency, Granularity, and Semantic Messaging

Commits act as the immutable ledger of a Data Scientist's thought process. Interns are expected to commit their code and notebook updates frequently, breaking down complex data transformations into logical, highly isolated units of work. A minimum of three to five meaningful commits per active development day is the expected baseline.

Commit messages must communicate the exact intent and context of the analytical change. Interns must utilize structured prefixes such as `data-clean:`, `eda:`, `feature-eng:`, or `model-tuning:` to maintain a highly readable repository history (e.g., `model-tuning: optimized XGBoost depth using TimeSeriesSplit`).

2. Jupyter Notebook Management and Experiment Tracking

Direct commits to the `main` or `master` branch are discouraged for major algorithmic changes. All new experiments (e.g., testing a new neural network architecture) should occur on an isolated feature branch. Furthermore, when committing Jupyter Notebooks (`.ipynb` files), interns should

ensure that output cells containing massive amounts of raw data or sensitive visualizations are cleared to prevent repository bloat.

3. Data Privacy and Credential Security

Security hygiene is paramount, especially when handling datasets mimicking citizen feedback or proprietary sales records. Interns must utilize `.gitignore` files to ensure that massive raw datasets (e.g., raw `.csv` or `.xlsx` files exceeding GitHub's recommended limits) or trained model weights (`.pkl` or `.h5` files) are not pushed to the repository directly. Hardcoding API keys, cloud access tokens, or database credentials within Python scripts or notebooks is grounds for immediate failure. All sensitive connection strings must be injected via secure Environment Variables.

By strictly enforcing these comprehensive specifications, the program ensures that deliverables transcend mere academic exercises, transforming into robust, scalable, and highly professional AI/ML portfolios indicative of elite industry standards.